

HLS Verification Report

Combined INT8 LeNet-5 Network — Real Vitis HLS C-Simulation vs. Python Reference
LeNet-5 FPGA Project · September 22, 2026

This report verifies that the combined INT8 LeNet-5 network, run through a real Vitis HLS C-simulation (not a software stand-in), produces layer-by-layer outputs matching an independent Python reference implementation. Every intermediate activation — from the first convolution through the softmax output — is compared stage by stage on a single MNIST test image.

1. The HLS Run

The combined 7-layer INT8 network was simulated with the `lenet5_top_int8_tb.cpp` testbench (`hls/lenet5_top_int8/tb/lenet5_top_int8_tb.cpp`), which chains all seven layers — C1, S2, C3, S4, C5, F6, and Output — and checks every intermediate INT8 stage against expected values baked in from the Python reference. Vitis HLS C-simulation was run on a Windows machine with Vitis HLS installed (this dev environment does not have Vitis HLS on PATH); the run reported **0 mismatches** across every layer and **TEST PASSED**.

The resulting csim dump files (per-layer HLS output text files under `lenet5_top_int8_proj/solution1/csim/build/`) were generated fresh on **September 22, 2026** and transferred back into this repository via git. Because C-simulation build products normally fall under this project's `**/*proj*/ gitignore` rule (they are ordinarily regenerated locally, not tracked), the dump files were force-added (`git add -f`) specifically to bring this one verification artifact set into version control for the record.

The layer-by-layer comparison below was produced by `hls/lenet5_top_int8/tb/compare_hls_vs_python.py`, which reads those HLS csim dumps and the Python int8 reference outputs (`lenet5_top_int8_test_data.h`) and diffs them stage by stage.

2. Layer-by-Layer Verification

Image: MNIST test[0], true label = 7. Comparison: lenet5_top_int8 C-simulation output vs. Python int8 reference.

Layer	Shape	Values	Max difference	Tolerance	Result
C1	28x28x6	4704	0	exact	MATCH
S2	14x14x6	1176	0	exact	MATCH
C3	10x10x16	1600	0	exact	MATCH
S4	5x5x16	400	0	exact	MATCH
C5	120	120	0	exact	MATCH
F6	84	84	0	exact	MATCH
Output	10	10	1.193e-07	1e-05	MATCH

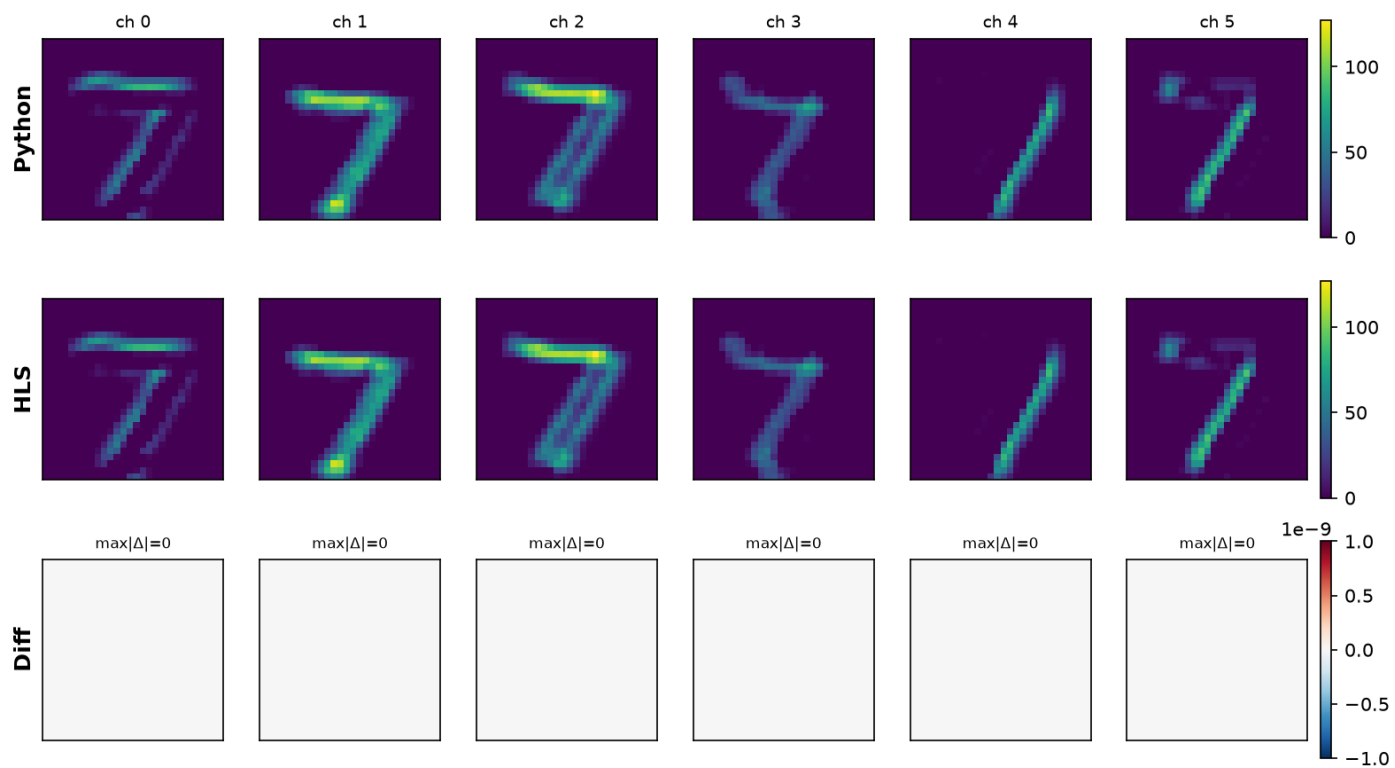
Predicted class (HLS)	Predicted class (Python int8 reference)	MNIST true label
7	7	7

3. Per-Layer Feature Map Comparisons

Each panel below shows the HLS csim output vs. the Python reference for every channel of that layer's activation map. C3 and S4 (16 channels each) are rendered in landscape orientation for legibility.

Layer C1 — HLS vs. Python Feature Maps

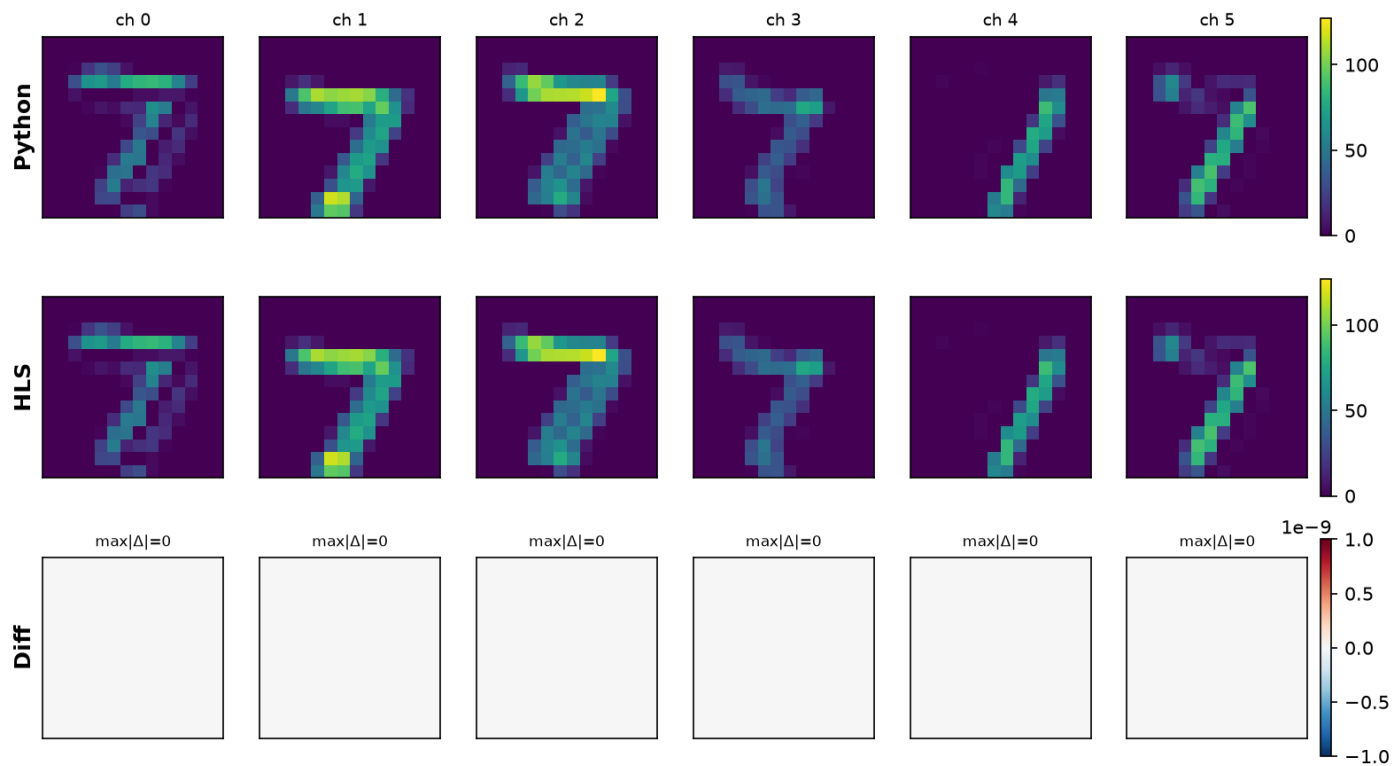
C1 — 6 channels, bit-exact



Layer C1: max diff = 0 (bit-exact)

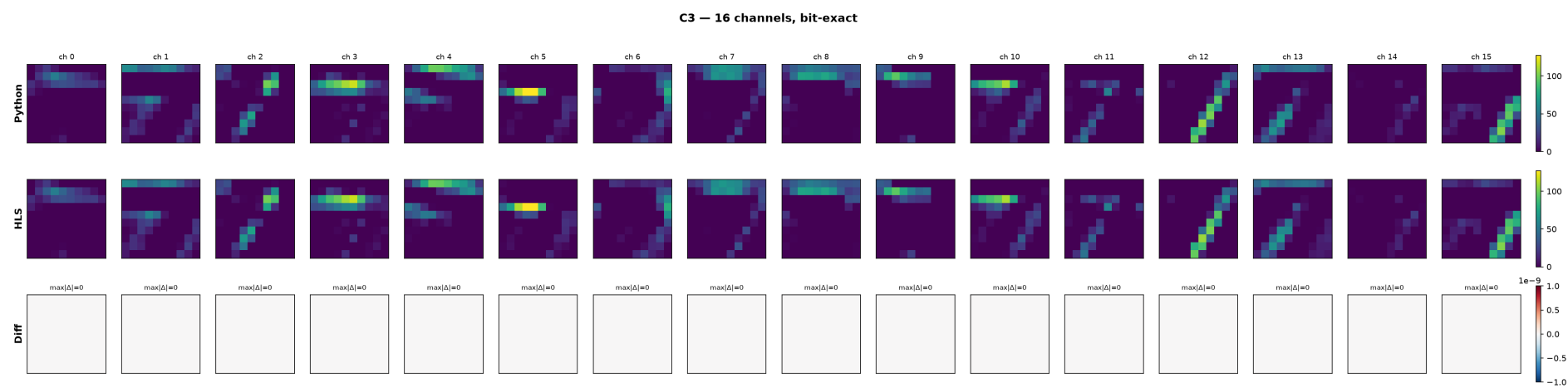
Layer S2 — HLS vs. Python Feature Maps

S2 — 6 channels, bit-exact



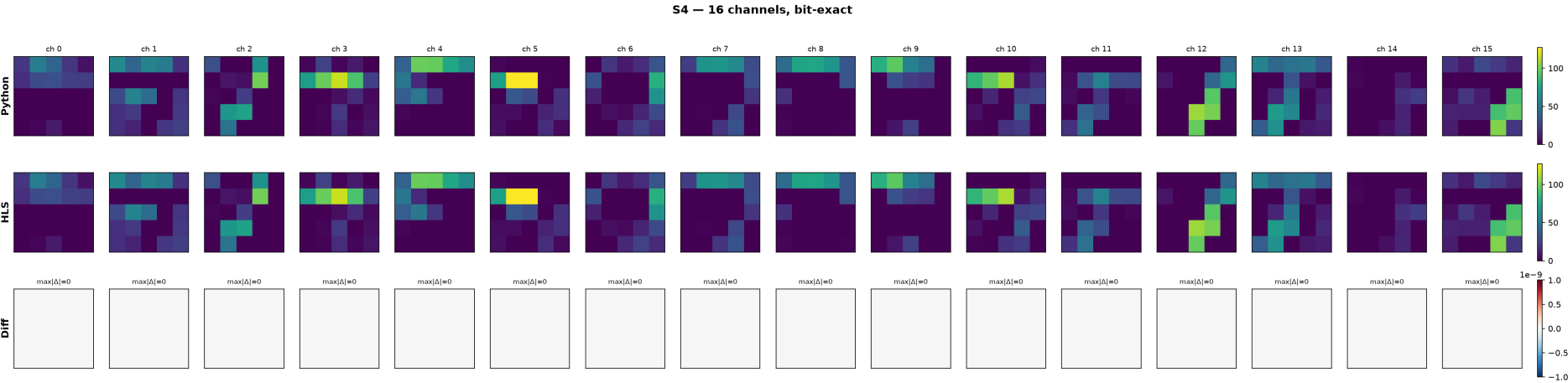
Layer S2: max diff = 0 (bit-exact)

Layer C3 — HLS vs. Python Feature Maps



Layer C3: max diff = 0 (bit-exact)

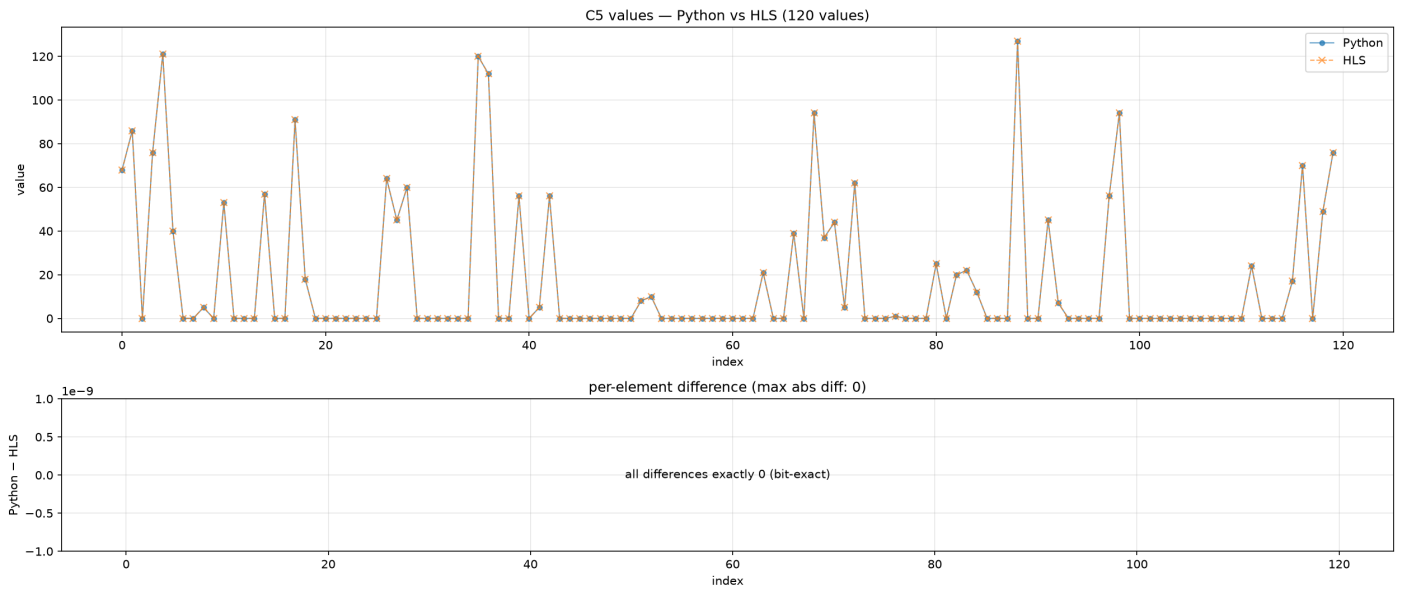
Layer S4 — HLS vs. Python Feature Maps



Layer S4: max diff = 0 (bit-exact)

Layer C5 — HLS vs. Python Feature Maps

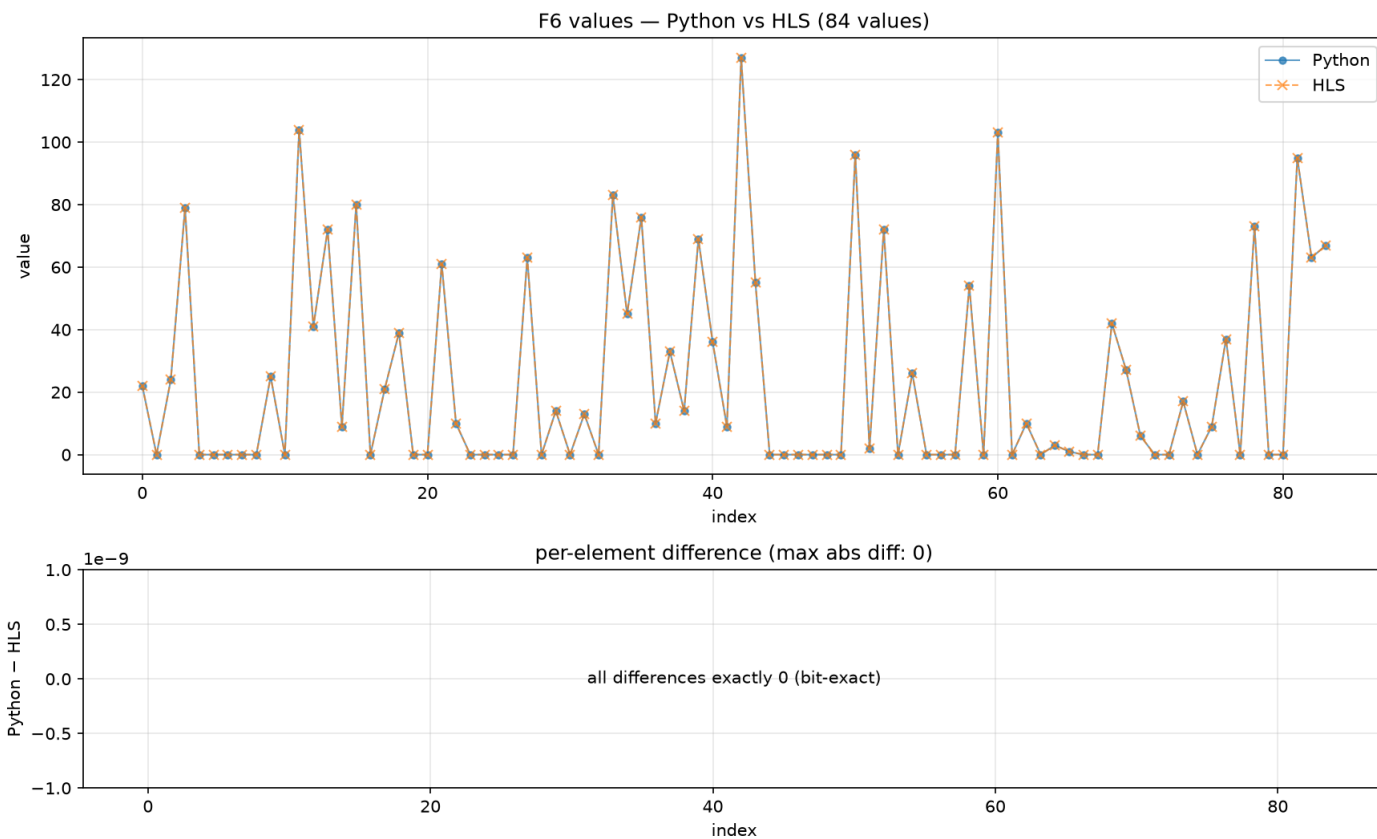
C5 — 120 values, max abs diff: 0.000e+00 (bit-exact)



Layer C5: max diff = 0 (bit-exact)

Layer F6 — HLS vs. Python Feature Maps

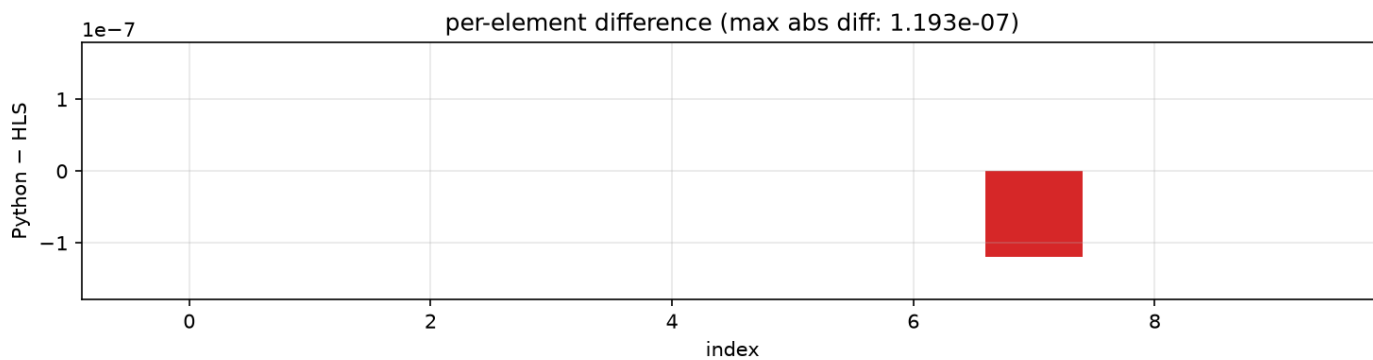
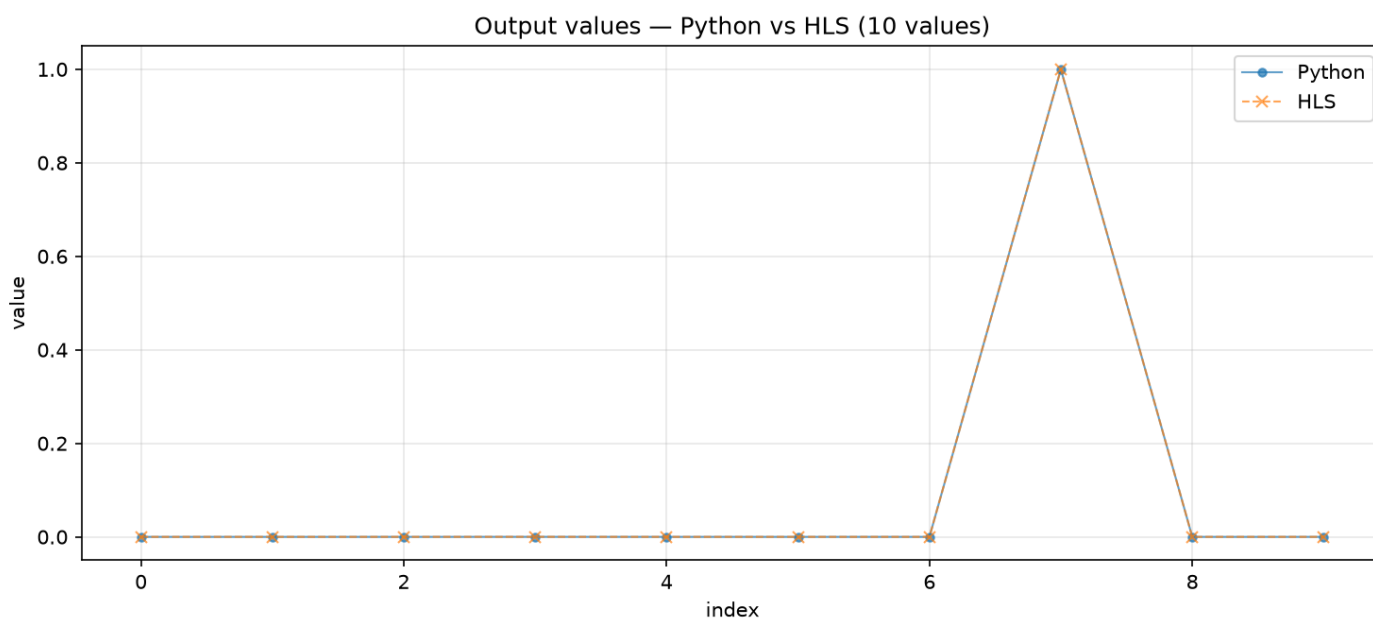
F6 — 84 values, max abs diff: 0.000e+00 (bit-exact)



Layer F6: max diff = 0 (bit-exact)

Layer Output — HLS vs. Python Feature Maps

Output — 10 values, max abs diff: 1.193e-07 (within 1e-05 tolerance)



Layer Output: max diff = 1.193e-07 (tolerance 1e-05)

All 7 layers verified against real synthesized-hardware-path (C-simulation) output: 6 of 7 bit-exact, Output within $1.19\text{e-}07$ (well under the $1\text{e-}5$ tolerance) due to expected floating-point softmax rounding.